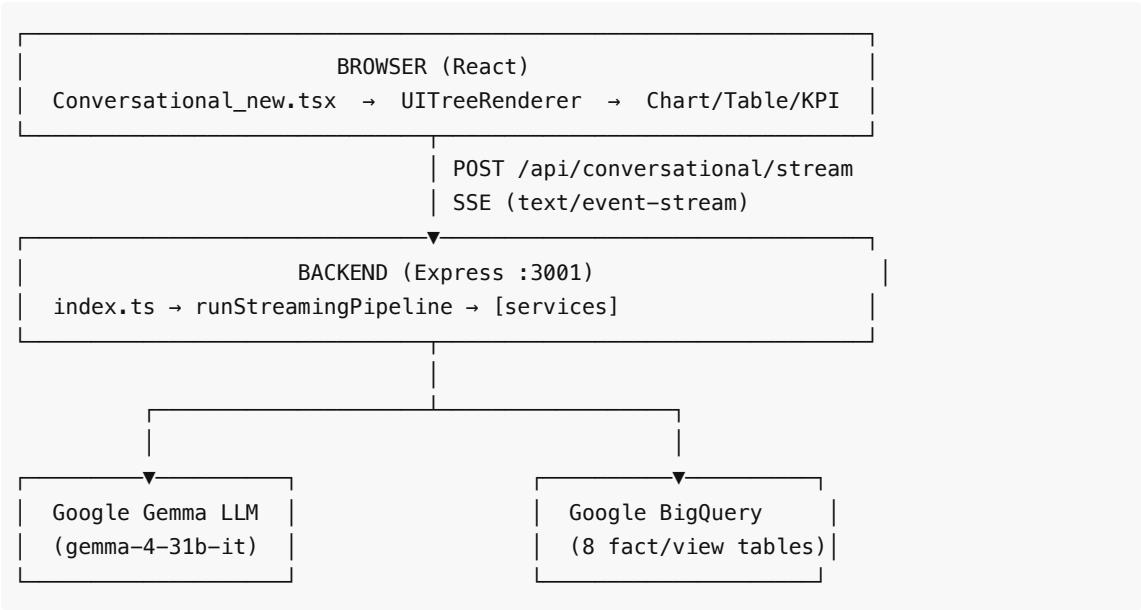


Chat Flow Architecture — conversationUI

Overview

End-to-end flow: user types query → SSE streaming pipeline → LLM classifies intent → BigQuery hydrates data → React renders components incrementally. Follow-up edits are handled with a fused single-LLM-call that classifies intent AND applies changes in one shot.

System Layers



1. Frontend Entry & Routing

File	Role
src/main.tsx	React DOM root mount
src/app/App.tsx	BrowserRouter — 15+ routes
src/app/pages/Conversational_new.tsx	Primary chat page

Key routes:

- `/conversational` — main chat UI
- `/report-flow` — guided report creation
- `/migration` — dataset migration assistant

2. Chat UI Component Tree

Conversational_new.tsx	
└─ ConversationSidebar	— list of saved conversations

```
├─ SuggestedPrompts          - context-aware quick prompts
├─ MessageList
│   └─ MessageBubble[]
│       ├── TextContent      - markdown-rendered text
│       ├── ClarificationCard - follow-up question from LLM
│       └─ UITreeRenderer    - generative charts/tables/metrics
│           ├── LineChart / BarChart / AreaChart / PieChart
│           ├── ComboChart / ScatterPlot / FunnelChart / HeatMap
│           ├── GaugeChart / Sparkline / ComparisonCard
│           ├── Table / GenerativeTable / PivotTable
│           ├── KPICard / KPIGrid / StatDelta
│           ├── RankedList
│           ├── InsightCard / SummaryText / AlertBanner / Callout / StepList
│           └─ TwoColumn / Section (layout wrappers)
├─ ChatInput                 - textarea + send button
└─ PersonaContext            - role-based quick actions
```

3. State Management

No external store (Redux/Zustand). All state lives in `Conversational_new.tsx` via `useState` .

Core Chat State:

State Variable	Type	Purpose
messages	Message[]	All messages in active conversation
conversations	Conversation[]	Sidebar conversation list
activeConversationId	string	Currently selected conversation
inputValue	string	Chat input field value
isGenerating	boolean	SSE stream in progress
flowState	'new' 'generating' 'clarifying' 'error'	Current UI flow state

Clarification Flow:

State Variable	Type	Purpose
clarificationContext	string	Current clarification question text
clarificationHistory	ClarificationTurn[]	Array of { question, answer } pairs

Report & Dataset Context:

State Variable	Type	Purpose
activeReportContext	object	Currently opened report data

selectedReport	object	Report selected for interaction
isReportPanelOpen	boolean	Report side panel visibility
activeDatasetContext	object	Currently opened dataset data
selectedDataset	object	Dataset selected for interaction
isDatasetPanelOpen	boolean	Dataset side panel visibility
activeTableRef	string	Active BQ table reference for follow-ups

Report Creation Flow (`createReportState`):

Field	Values
step	data_source marketplace_dataset_grid intent dimensions metrics usage layout visualization preview execution_routing review null
dataSource	Selected source type
selectedMetrics	Chosen metric fields
selectedDimensions	Chosen dimension fields
expectedUsage	User consumption expectations
layoutType	template custom reference
selectedTemplate	Template name
customLayoutComponents	Components for custom layout
referenceReportLink	URL for reference layout

Context:

- [src/app/context/PersonaContext.tsx](#) — persona (`marketing_director` / `power_user`) drives quick actions + intent cards

4. Message Send Flow (Frontend)



```

conversationHistory)
  |
  ▼
fetch POST /api/conversational/stream
  |
  ▼
streamReader loop (ReadableStream)
  |
  ├── event: status          → show progress indicator ("Understanding query...",
  "Querying BigQuery...")
  ├── event: meta            → set message title/description/template
  ├── event: component       → push UINode to message.components[]
  ├── event: followUp        → set suggestedPrompts[] (label + intent)
  ├── event: clarification   → set flowState='clarifying', show ClarificationCard
  ├── event: qa_answer       → show text answer without new dashboard
  ├── event: acknowledgment → confirm edit was applied
  ├── event: bq_debug        → optional BQ query metadata (debug only)
  ├── event: done            → finalize stream, set isGenerating=false
  └── event: error           → set flowState='error', show error message
  |
  ▼
setMessage() – incremental patch per event
  |
  ▼
UITreeRenderer renders each component as it arrives

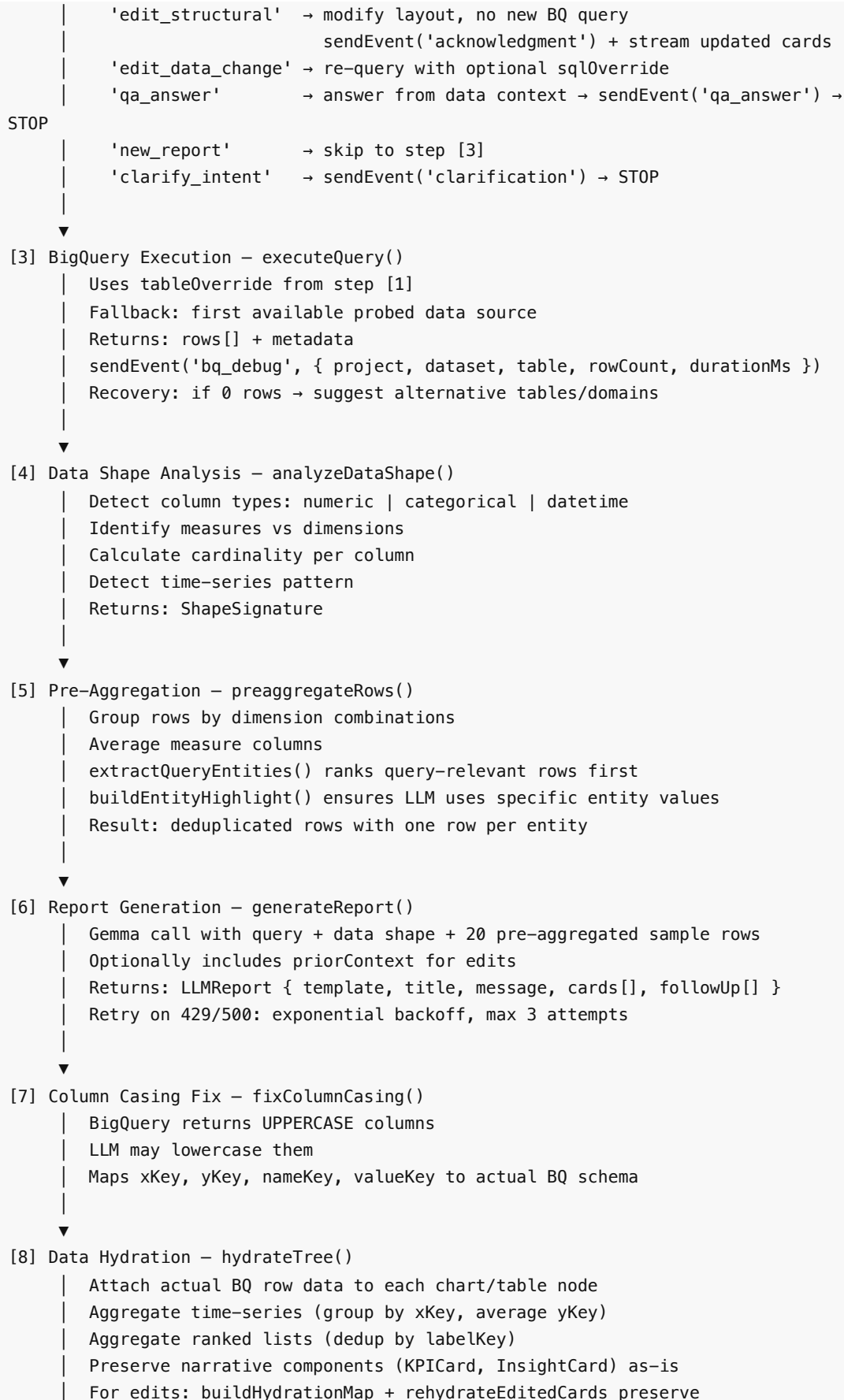
```

5. Backend Pipeline — `runStreamingPipeline.ts`

```

POST /api/conversational/stream
  |
  ▼
[0] Check cache (cacheService, 5-min TTL)
  |   Key = hash of (query + persona + clarificationHistory + priorContext)
  |   hit → stream cached components
  |   miss ↓
  |
  ▼
[1] Intent Classification – analyzeQuery()
  |   Fast path: extractContextFromText() checks for exact domain/report name
  |   LLM path:  buildAnalyzePrompt → Gemma call (JSON mode, temp=0.2)
  |   Returns action: 'route' | 'clarify'
  |
  ├── if 'clarify' → sendEvent('clarification', { opener, question, options }) →
STOP
  |
  ▼
[2] Follow-Up Edit Handler – classifyAndEditReport()
  |   ONLY runs if: hasExistingReport && !inClarificationFlow && !skipClarification
  |   Single LLM call classifies AND applies changes
  |   Returns action:

```



```

| original data arrays when LLM modifies card structure
|
▼
[9] Stream & Cache
| sendEvent('meta',      { title, description, template, activeTable })
| sendEvent('component', hydratedNode) ← one per card
| sendEvent('followUp',  [{ label, intent }])
| sendEvent('done',      { success: true })
└─ cache result (5-min TTL)

```

6. SSE Event Protocol

All events sent as `text/event-stream` from Express to the browser fetch reader.

```

event: status
data: { "message": "Understanding your query..." }

event: meta
data: { "title": "...", "description": "...", "template": "trend_analysis",
"activeTable": "fact_sug_monthly_rollup" }

event: component
data: { "renderType": "LineChart", "props": {...}, "data": [...] }

event: followUp
data: { "prompts": [{ "label": "Show by region", "intent": "comparison" }] }

event: clarification
data: { "opener": "...", "currentQuestion": { "question": "...", "options": [...],
"..."] }, "isRecovery": false }

event: qa_answer
data: { "message": "Revenue was $4.2M in Q3...", "followUp": [...] }

event: acknowledgment
data: { "message": "Done – removed the bar chart and added a table." }

event: bq_debug
data: { "project": "...", "dataset": "...", "table": "...", "rowCount": 120,
"durationMs": 340 }

event: error
data: { "message": "..." }

event: done
data: { "success": true }

```

Frontend parses each chunk:

```

const [eventLine, dataLine] = chunk.split('\n')
const eventType = eventLine.replace('event: ', '')

```

```
const payload = JSON.parse(dataLine.replace('data: ', ''))
```

7. LLM Integration — `llmHandler.ts`

Four LLM call types:

Function	When	Model Input	Output
<code>analyzeQuery()</code>	Every new message	query + clarification history + catalog context	<code>AnalyzeResult</code> (route or clarify)
<code>generateReport()</code>	After BQ data fetched	query + data shape + 20 pre-aggregated rows + <code>priorContext</code>	<code>LLMReport</code> (cards, <code>followUp</code> , title, template)
<code>classifyAndEditReport()</code>	Follow-up on open report	query + <code>currentCards</code> + <code>priorContext</code> + <code>dataContext</code> + <code>conversationHistory</code>	<code>FusedIntentResult</code> (edit type + applied changes)
<code>callLLM()</code>	<code>/api/chat</code> endpoint	system prompt + messages	{ message, cards, <code>followUp</code> }

Result Types:

```
type AnalyzeResult =
  | { action: 'clarify'; opener: string; question: string; options: string[] }
  | { action: 'route'; table: string; intent: 'trend' | 'comparison' | 'metric_by_dimension' }

type FusedIntentResult =
  | { action: 'new_report' }
  | { action: 'edit_data_change'; sqlOverride?: string }
  | { action: 'edit_structural'; acknowledgment: string; title: string; message: string; cards: ReportCard[]; followUp: FollowUp[] }
  | { action: 'qa_answer'; message: string; followUp: FollowUp[] }
  | { action: 'clarify_intent' }
```

Retry logic:

```
attempt 1 → if 429 or 500 → wait 1s → attempt 2 → wait 2s → attempt 3 → throw
```

Startup probe: `probeTableAvailability()` queries BigQuery at startup and every 24 hours. Only tables confirmed to exist are offered in clarification options.

8. BigQuery Data Sources — `dataSourceMap.ts`

Table	Domain	Report Name	Primary Metrics
<code>fact_sug_monthly_rollup</code>	Sales	Monthly Revenue & Take Rate	Revenue, Run Rate, Take Rate %, Return Rate %, ...

			AARD %, RIS %
v_monthly_territory_performance	Sales	Territory Performance Scorecard	Performance Score, Territory Rank, Revenue
v_daily_sales_detail	Sales	Daily Sales Detail	Units Sold, Revenue, Outlet
fact_sug_monthly_rollup	Network	Churn & Retention Metrics	Return Rate %, RIS %, AARD %, Retention Index
fact_network_kpi_points	Network	Network KPI Trends	Network KPI Score, Signal Strength, Outage Count, Latency
fact_dynamic_scores	Network	Dynamic Score Rankings	Score, Rank, Performance Index
fact_contact_center_metrics	Contact Center	Agent Performance Report	Box Close %, AHT, Transfer %, Sales Time %
fact_sug_monthly_rollup	Customer Experience	Customer Retention Analysis	Return Rate %, RIS %, AARD %, Territory Revenue

Catalog metadata tables: `catalog_reports` , `catalog_datasets` .

9. UITreeRenderer — Component Dispatch

```
UITreeRenderer receives { renderType, props, data }  
  
|  
|— Metric Components —————  
| "KPICard"      → Metric card with trend badge  
| "KPIGrid"      → Grid of KPICards  
| "StatDelta"    → Current vs previous value comparison  
| "GaugeChart"   → Progress toward target (red/amber/green zones)  
|  
|— Chart Components —————  
| "LineChart"    → Time-series trend (Recharts)  
| "BarChart"     → Categorical breakdown (Recharts)  
| "AreaChart"    → Time-series with fill (Recharts)  
| "PieChart"     → Proportion breakdown (Recharts)  
| "ComboChart"   → Dual-axis bar + line  
| "ScatterPlot"  → Correlation (xKey, yKey, zKey)  
| "FunnelChart"  → Conversion funnel  
| "HeatMap"      → Time-of-day / day-of-week patterns  
| "RankedList"   → Top-N items with bar indicators  
| "Sparkline"    → Tiny inline KPI + trend line  
| "ComparisonCard" → Head-to-head entity comparison  
|
```

— Table Components —	
"Table"	→ Sortable data grid
"GenerativeTable"	→ Interactive table (sort/filter)
"PivotTable"	→ Cross-tab breakdown
— Narrative Components —	
"InsightCard"	→ Key finding or insight
"SummaryText"	→ Paragraph of explanatory text
"AlertBanner"	→ Alert (info / warning / error / success)
"Callout"	→ Highlighted key finding
"StepList"	→ Numbered action items
— Layout Components —	
"TwoColumn"	→ Two children side-by-side
"Section"	→ Grouped content with optional title

All chart types use design system tokens from MASTER.md (warm off-white surface, brand accent #D4572A , category colors).

10. Multi-Turn Clarification Flow

```

User query is ambiguous
|
▼
analyzeQuery() returns action: 'clarify'
Options sanitized against probed available BQ tables
|
▼
Backend sends event: clarification
{ opener, currentQuestion: { question, options }, isRecovery? }
|
▼
Frontend sets flowState = 'clarifying'
Renders ClarificationCard with question + option chips
|
User selects option or types answer
|
▼
handleClarificationResponse(answer)
|  → appends { question, answer } to clarificationHistory[]
|  → calls sendToLLM() again with full clarificationHistory
|
▼
Pipeline resumes from step [1] with enriched context

```

11. Follow-Up / Edit Flow

```

Report rendered on screen
|

```

User sends follow-up ("show by region", "answer a question about this data")

|



handleAsk() includes currentCards + conversationHistory in request

|



Backend: classifyAndEditReport() – single fused LLM call

|

├ 'edit_structural':

LLM modifies UITypeTree layout in-place

No new BigQuery call

buildHydrationMap + rehydrateEditedCards re-attach original data

sendEvent('acknowledgment') + stream updated cards

|

├ 'edit_data_change':

Optional sqlOverride from LLM

New BigQuery query

Full pipeline from step [3] runs

|

├ 'qa_answer':

buildCompactDataContext extracts readable metric values

LLM answers question using existing data

sendEvent('qa_answer') – no new dashboard rendered

|

└ 'new_report':

skip to step [3] – full new pipeline run

12. Caching — `cacheService.ts`

Key = stable hash of: userQuery + persona + clarificationHistory + priorContext

TTL = 5 minutes (default), 10 minutes (catalog)

On hit: skip steps [3]–[8], stream cached components

On miss: run pipeline, store result after step [9]

Catalog refreshes every 24 hours at server startup via `catalogRefresher.ts`. Catalog = markdown of all BigQuery domains, table schemas, report inventory — injected as LLM system context.

13. Conversation Persistence

Conversation

├ id: uuid

├ title: string (auto-generated from first query)

├ status: 'draft' | 'active' | 'planned'

├ messages: Message[]

| └ role: 'user' | 'assistant'

| └ content: string

| └ components: UINode[] ← hydrated chart/table data

```
| └─ suggestedPrompts: Array<{ label, intent }>
| └─ createdAt / updatedAt
```

Stored in component state; sidebar lists all conversations. Selecting a conversation restores the full message history including rendered components. Last 6 turns of conversation history are passed to `classifyAndEditReport()` on every follow-up.

14. Error Handling

Layer	Error	Handling
LLM quota	429	Retry x3 exponential backoff
LLM server error	500	Retry x3 exponential backoff
LLM empty response	Empty cards	Fallback card generator runs
BQ query failure	SQL error	event: error to frontend
BQ zero rows	No data	Recovery: suggest alternative tables/domains
Column mismatch	LLM lowercase vs BQ uppercase	<code>fixColumnCasing()</code> normalizes
Network drop	SSE disconnect	Frontend shows error state, retry button

15. Persona-Based Customization

`PersonaContext` provides role-specific defaults:

Persona	Default Metrics	Quick Actions
marketing_director	channel ROI, leads, campaign performance	"Show pipeline", "Compare channels"
power_user	all available, raw data access	"Run custom query", "Export BQ SQL"

Injected into LLM system prompt so generated reports prioritize relevant metrics per role.

Key File Index

File	Purpose
src/app/pages/Conversational_new.tsx	Chat UI + all frontend state
src/app/components/UITreeRenderer.tsx	Renders all generative components (20+ types)
src/app/context/PersonaContext.tsx	Role-based context

backend/src/index.ts	Express server + route definitions
backend/src/pipeline/runStreamingPipeline.ts	Master orchestrator (10-step pipeline)
backend/src/pipeline/runPipeline.ts	Non-streaming fallback pipeline
backend/src/services/llmHandler.ts	Gemma LLM calls + fused intent classification
backend/src/services/queryEngine.ts	BigQuery SQL builder + executor
backend/src/services/dataShapeAnalyzer.ts	Column type + cardinality analysis
backend/src/services/dataSourceMap.ts	8 BigQuery table/domain definitions
backend/src/services/cacheService.ts	TTL in-memory cache
backend/src/services/catalogRefresher.ts	24h catalog refresh
backend/src/services/intentClassifier.ts	Legacy keyword classifier (superseded by llmHandler.analyzeQuery)
backend/src/lib/bigqueryClient.ts	BQ connection + query execution
backend/src/types/index.ts	Shared TypeScript interfaces